How to Quantify Scalability

Synopsis of The Universal Scalability Law (USL)

Neil J. Gunther

Updated on May 10, 2025

The purpose of models is not to fit the data but to sharpen the questions. — Sam Karlin

It's easy to find discussions on the Internet about how to improve scalability performance; especially for software applications. It's rare, however, to find such discussions that actually **quantify** the claimed scalability improvement. More often, they just read like,"Do this FTW" recipes.

This page is intended to supplement the following Chapters in my *Guerrilla Capacity Planning* book [Gunther 2007]:

Chap. 4: Scalability: A Quantitative Approach

Chap. 5: Evaluating Scalability Parameters

Chap. 6: Software Scalability

as well as the notes associated with my *Guerrilla Capacity and Performance* (GCAP) training class.

It's also an attempt to provide a quick overview of the USL **methodology**, including the **latest developments** since the book, including:

1. Applying the USL to distributed systems
2. The three-parameter version of the USL
3. Superlinear scaling in Hadoop

My original 1993 paper introduced the USL under the name "Super Serial" model because, at that time, I viewed it as an extension of the *serial fraction* concept that underpins Amdahl's law. The idea of using the USL as a **statistical regression** model came much later.

Contents

1 Universal Scalability Law (USL) 3

1.1 The Scalability Model . . . 3

1.2 The Three Cs . . . 5

1.3 Theoretical Justification . . . 6

1.4 Applicability . . . 6

2 How to Use It 7

2.1 Virtual load testing . . . 7

2.2 Detecting bad measurements . . . 7

2.3 Performance heuristics . . . 8

2.4 Performance diagnostics . . . 8

2.5 Production environments . . . 8

2.6 Scalability Zones . . . 8

2.7 My Blog . . . 9

3 Tools for Using USL 9

3.1 Excel Spreadsheet . . . 9

3.2 OpenOffice Spreadsheet . . . 9

3.3 R Packages and Scripts . . . 10

4 Presentations 10

4.1 DSconf 2019 . . . 10

4.2 ACM 2018 . . . 10

4.3 ACM 2015 . . . 11

4.4 Usenix LISA 2014 . . . 11

4.5 Hotsos 2011 . . . 12

4.6 Surge 2010 . . . 12

4.7 Velocity 2010 . . . 13

4.8 Ignite! 2009 . . . 13

5 Bibliography 14

1 Universal Scalability Law (USL)

The original derivation of the *Universal Scalability Law*, or USL, was presented at the 1993 CMG conference [Gunther 1993]. A brief account of its application to parallel processing performance (under the name *super-serial model*) was presented in Chaps. 6 and 14 of my first book [Gunther 1998]. A more complete derivation with example applications is presented in Chaps. 4–6 of my GCaP book [Gunther 2007]. The supporting mathematical theorems (see Section 1.3) are presented in papers cited in the Bibliography (see Section 5).

Some reasons why you should understand the USL include:

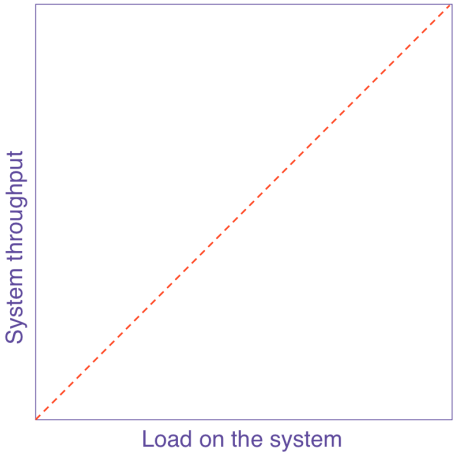
1. A lot of people use the term "scalability" without clearly defining it, let alone defining it quantitatively. Computer system scalability must be quantified. If you can't quantify it, you can't guarantee it. The *universal law of computational scaling* provides that quantification.

2. One of the greatest impediments to applying queueing-theory models (whether analytic or simulation) is the inscrutibility of service times within an application. Every queueing facility in a performance model requires a service time as an input parameter. Without the appropriate queues in the model, system performance metrics like throughtput and response time, cannot be predicted. The USL leapfrogs this entire problem by NOT requiring ANY low-level service time measurements as inputs.

1.1 The Scalability Model

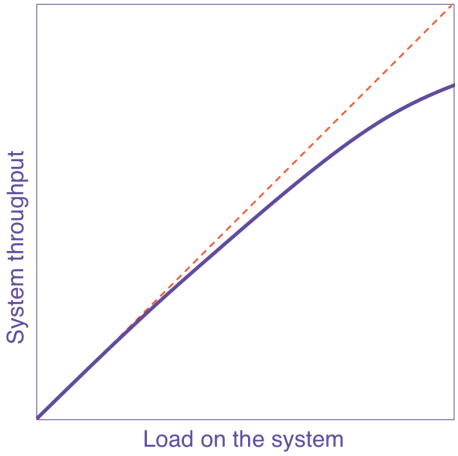
Defining the system throughput $X(N)$ at a given load, N, the fundamental scaling effects contained in the **USL (Universal Scalability Law)** can be depicted schematically as

A. Equal bang for the buck



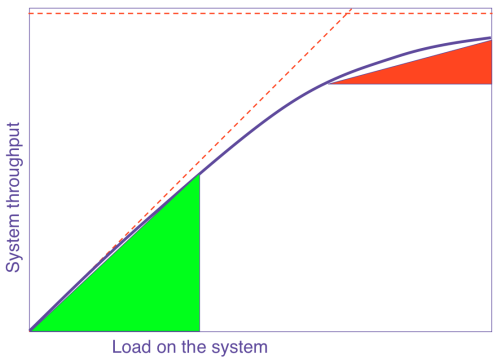
$\alpha = 0,\ \beta = 0$

B. Cost of sharing resources



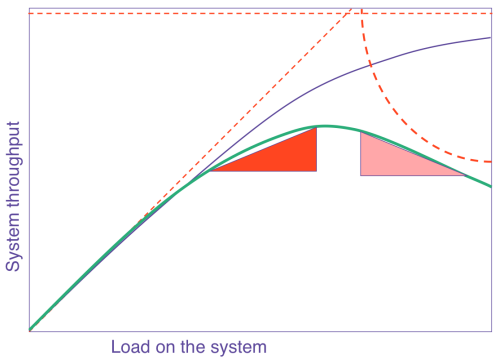
$\alpha > 0,\ \beta = 0$

C. Diminishing returns from contention



$\alpha \gg 0,\ \beta = 0$

D. Negative returns from coherency delay



$\alpha \gg 0,\ \beta > 0$

Each of these contributions can be combined into a single equation to produce an analytic **scalability model**, viz., the USL. The following version of the USL appears in Guerrilla Capacity Planning book .

$$C(N) \;=\; \frac{N}{1+\alpha(N-1)+\beta N(N-1)} \tag{1}$$

Here, $C(N) = X(N)/X(1)$ is the **relative capacity** (or normalized throughput) at each successive load point, $N = 1,2,3,\ldots$. Conversely, any throughput value, *including those that have not been measured*, can be expressed in terms of $C(N)$ as the proiduct

$$X(N) = C(N) \cdot X(1). \tag{2}$$

One drawback of eqn. (1) is that it requires measuring the value of $X(1)$ to do the normalization. Very often, however, we don't know the value of $X(1)$ because it was not measured or it **cannot** be measured. In that case, the value of $X(1)$ has to be **estimated** by interpolating the existing data. A method for doing that is discussed in [Gunther 2007].

A generalization of eqn. (1) introduces a third modeling parameter, γ, into the numerator, such that

$$X(N) \;=\; \frac{\gamma\, N}{1+\alpha(N-1)+\beta N(N-1)}. \tag{3}$$

Notice that the left side of eqn. (3) is now $X(N)$ (the throughput), not $C(N)$ (the relative capacity). The reasoning behind this modification can be found in my blog post [Gunther 2018]. A significant virtue of this approach is that it makes it easier to apply the USL to the available data because the estimation of $X(1)$ gets incorporated into the same nonlinear statistical regression procedure that is already being used to estimate the α, β parameters. The regression calculation doesn't care whether you have 2 or 3 parameters.

The applied load might be induced either by N virtual users e.g., generated by a load testing tool like JMeter, or correspond to real users on a production platform. In the parallel processing literature, the **normalized execution time**, $S(N) = T(1)/T(N)$, known as the **speedup**, is often used in place of the relative capacity $C(N)$, $C(N)$ is the dual representation of $S(N)$.

1.2 The Three Cs

The coefficients, α, β, γ, in eqn.(3) can be identified respectively with the following fundamental scalability attributes [Gunther 2018]:

1. **Concurrency** or ideal parallelism (with proportionality parameter γ), which can be interpreted as either:
   (a) the slope associated with linear parallelism, i.e., the straight line $X(N) = \gamma N$ in Fig. A. In that case other USL coefficients are $\alpha = \beta = 0$.
   (b) the maximum throughput attainable with a single load generator ($N = 1$), i.e., $X(1) = \gamma$.
2. **Contention** (with proportionality parameter α) due to waiting or **queueing** for shared resources.
3. **Coherency** (with proportionality parameter β) due to the delay for data copies on different **distributed** resources to become consistent (e.g., cache coherent), with one another. The USL model assumes this is accomplished via **point-to-point exchange**. Hence, the N^2 term in the denominator of eqn.(3).

Remark 1. *When $\beta = 0$ and $\gamma = 1$, eqn. (3) reduces to* **Amdahl's law**. *See theorem 1.*

The independent variable, N, can represent either

Software Scalability: Here, the number of **virtual users** or load generators (N) is incremented on a fixed hardware configuration. In this case, the number of users acts as the independent variable while the processor configuration remains fixed over the range of user-load measurements. This is the most common situation found in load testing environments where tools like LoadRunner or JMeter are used.

Hardware Scalability: Here, the number of **physical processors** (N) is incremented in the hardware configuration while keeping the user load per processor fixed. In this case, the number of users executing per processor (e.g., 100 users per processor) is assumed to remain the same for every added processor. For example, on a 32 processor platform you would apply a load of $N = 3200$ users to the test platform.

Equation (3) tells us that hardware and software scalability are really just two sides of the same coin: something not generally recognized. A non-zero value of β is associated with measured throughput that goes retrograde, i.e., decreases with increasing load or configuration size.

Remark 2. *The goal of using eqn.(3) is not to produce a curve that passes through every data point. That's called spline fitting and that's what graphics artists do. It's not statistical analysis.*

The fitted coefficients in eqn.(3) then provide an estimate of the user load at which the maximum scalability will occur in figure D:

$$N_{max} = \sqrt{1 - \frac{\alpha}{\beta}} \tag{4}$$

Notice that the γ parameter plays no role in eqn. (4). That's because it locates the position of the maximum in the USL curve on the x-axis, whereas γ scales the y-axis values. The corresponding maximum throughput X_{max} is found by substituting N_{max} into eqn. (3):

$$X_{max} = X(N_{max}) \tag{5}$$

Scalability profiles in Figures B and C (above) correspond to $\beta = 0$, which means that N_{max} occurs at infinity. Put differently, the scalability curve simply approaches a **ceiling** at $1/\alpha$ (the horizontal dashed line in the plots). See Chapter 6 *Software Scalability* in [Gunther 2007].

1.3 Theoretical Justification

The following theorems provides the justification for applying the same USL eqn.(3) to both **hardware** and **software** systems. Theorem 1 establishes the queue-theoretic basis for Amdahl's law [Gunther 2002].

Theorem 1 (Gunther (2002))**.** *Amdahl's law for parallel speedup is equivalent to the synchronous queueing bound on throughput in the machine repairman model of a multiprocessor.*

Similarly, theorem 2 establishes the queue-theoretic basis for the USL.

Theorem 2 (Gunther (2008))**.** *The USL is equivalent to the synchronous queueing bound on throughput for a linear load-dependent machine repairman model of a multiprocessor.*

The proof can be found in [Gunther 2008].

Amdahl's law is therefore subsumed by the USL because it corresponds to the case $\gamma = 1, \beta = 0$. Both Amdahl's law and the USL belong to a class of mathematical functions called rational functions. Moreover, these theorems have also been confirmed experimentally using **event-driven simulations** [HoltGun 2008]. In other words, the whole USL approach to quantifying scalability rests on a fundamentally sound *physical* footing.

1.4 Applicability

The USL model has wide-spread applicability, including:

- Load testing tools like LoadRunner and JMeter. See section 2.1.

- Modeling disk arrays, SANs, and multicore processors.
- Modeling distributed application software, e.g., Hadoop. (See section 4.3)
- Accounts for such effects as memory thrashing, and cache-miss latencies.

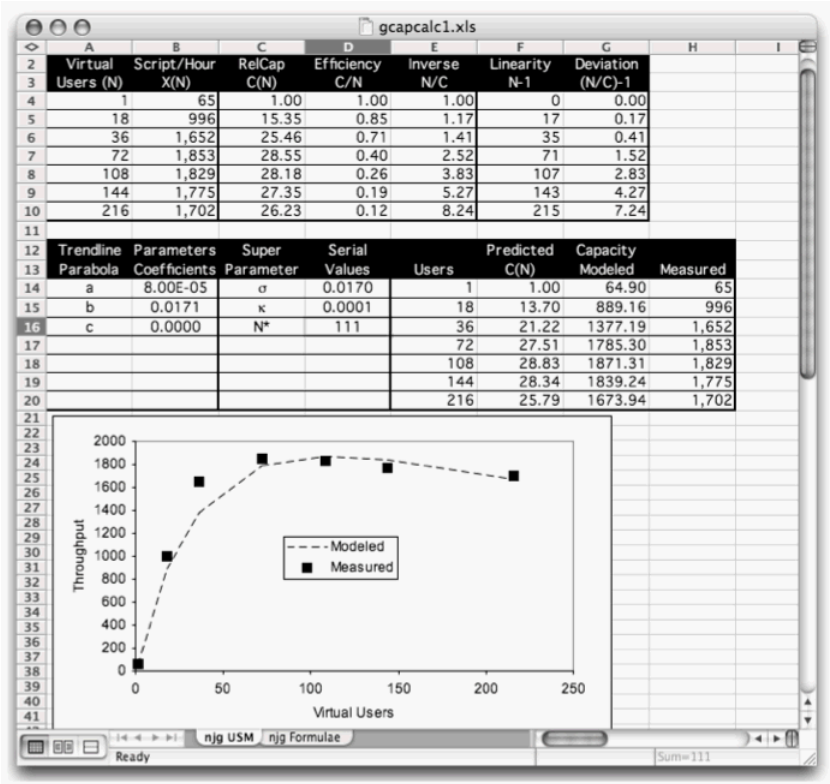
That's why it's called *universal*. ☺

2 How to Use It

This section offers some ideas for how to apply the USL.

2.1 Virtual load testing

The USL in eqn.(3) allows you take a sparse set of load measurements—at least **half a dozen** load points—-and from those data, determine how your application will scale under user-loads that are larger than may be cost-effective to generate on your test rig. The USL analysis can all be done in a spreadsheet app like, Excel or LibreOffice.



Virtual Users (N)	Script/Hour X(N)	RelCap C(N)	Efficiency C/N	Inverse N/C	Linearity N-1	Deviation (N/C)-1
1	65	1.00	1.00	1.00	0	0.00
18	996	15.35	0.85	1.17	17	0.17
36	1,652	25.46	0.71	1.41	35	0.41
72	1,853	28.55	0.40	2.52	71	1.52
108	1,829	28.18	0.26	3.83	107	2.83
144	1,775	27.35	0.19	5.27	143	4.27
216	1,702	26.23	0.12	8.24	215	7.24

Trendline Parabola	Parameters Coefficients	Super Parameter	Serial Values	Users	Predicted C(N)	Capacity Modeled	Measured
a	8.00E-05	σ	0.0170	1	1.00	64.90	65
b	0.0171	κ	0.0001	18	13.70	889.16	996
c	0.0000	N*	111	36	21.22	1377.19	1,652
				72	27.51	1785.30	1,853
				108	28.83	1871.31	1,829
				144	28.34	1839.24	1,775
				216	25.79	1673.94	1,702

2.2 Detecting bad measurements

Equation (3) is not a crystal ball. It cannot foretell the onset of broken measurements or intrinsic pathologies. When the data diverge from the model, that does not automatically make the model wrong. You need to stop measuring and find out why.

2.3 Performance heuristics

The relative sizes of the α and β parameters tell you respectively, whether contention effects or coherency effects are responsible for poor scalability.

2.4 Performance diagnostics

What makes (3) easy to apply, also limits its diagnostic capability. If the parameter values are poor, you cannot use it to tell you what to fix. All that information is in there alright, but it's compressed into the values of those two little parameters. However, other people e.g., application developers (the people who wrote the code), the systems architect, may easily identify the problem once the universal law has told them they need to look for one.

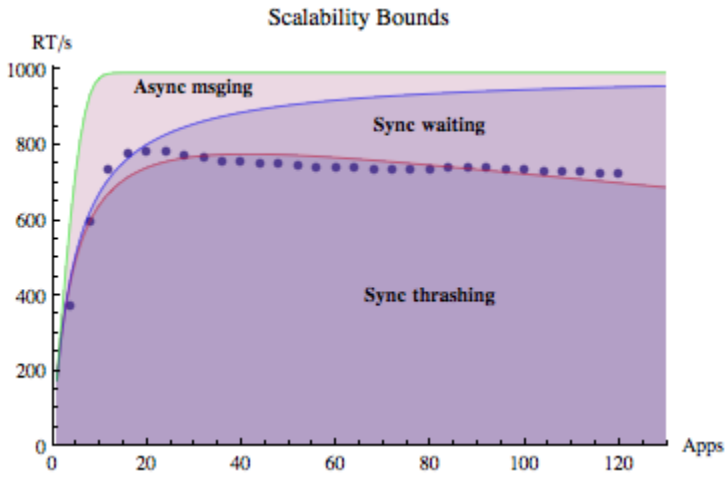
2.5 Production environments

The USL can be applied to performance data collected from production environments. See Linux-Tomcat Application Performance on Amazon AWS for a detailed example.

The main issue is determining the appropriate **independent** variable, e.g., N users or processes, not dependent variables like utilization $\rho(N)$. Then you only need $X(N)$ data as the **dependent** variable to regress against. Those $X(N)$ values should be determined from data that are collected during a measurement window with as few large-transient effects as possible, i.e., close to steady state. From there, everything should go as per the usual procedure described in Section 2.

2.6 Scalability Zones

In 2008, I introduced the concept of *scalability zones* [HoltGun 2008]. The idea is, instead of just considering a simple curve fit to the data points, we let the data fall across a number of regions whose boundaries are defined by a *set* of fitted scalability curves. The boundaries define *zones* as shown below. Moreover, these zones can be directly indentified with queueing effects and this can be immensely helpful to apps developers looking for ways to tweak their code to improve performance.



In this example, the plot shows the measured scalability (dots) of WebSphere MQ V6.0 nonpersistent 2KB messages (in Rnd Trips/sec) as a function of the increasing number of driving applications (Apps). The colored regions traversed by the data (dots) indicate the kind of synchronous queueing effects responsible for the apparent loss of scalability above about $N = 15$ Apps.

2.7 My Blog

Various subtle points about using the USL are covered in my blog postings and the associated comments.

3 Tools for Using USL

Some tools are already available for applying the USL model to the estimation of both **hardware** (p) and **software** (N) scalability. These are the same tools as used in my classes on Guerrilla Capacity Planning and Guerrilla Data Analysis Techniques.

Most people will probably find it convenient to start with an Excel (see Section 3.1) or OpenOffice (see Section 3.2) spreadsheet. For serious capacity and performance analysis, however, beware potential numerical precision problems when using those tools.

Those with a more sophisticated statistical and programming background may find the R packages listed in Section 3.3 to be more appropriate.

3.1 Excel Spreadsheet

Excel spreadsheet USLcalc.xls.

> Note that this Excel file contains the reorganized version of the USL equation as it appears in the *Guerrilla Capacity Planning* book:
>
> 1. Current **hardware** equation (5.1) which uses the coefficients labeled σ and κ.
> 2. Current **software** equation (6.7) which uses the coefficients labeled α and β.
>
> The difference in the USL versions (old and current) has to do with the way the coefficients are included in the USL equation and therefore how they are calculated from the fitting parameters **a** and **b** in Excel. This only applies to the Excel version of the USL, where an inversion transformation is necessary because Excel cannot fit a rational function by default. Tools like R and Mathematica do not have this limitation.

3.2 OpenOffice Spreadsheet

An OpenOffice version of USLcalc.xls, called USLcalc.ods, has been provided compliments of a GCaP guerrilla. It compensates for the lack of a polynomial fitting routine in ODS.

3.3 R Packages and Scripts

1. The R script USLcalc.r uses the `nls()` function to perform nonlinear regression with the USL model (2009).
2. CRAN package USL: Analyze system scalability with the Universal Scalability Law by Stefan Moeding (2013). Install the package from the R-Console using the command `install.packages("usl")`
3. RForge package SATK: Scalability Analysis Toolkit by Paul Puglia (2013). Install the package from the R-Console using the command `install.packages("SATK", repos="http://R-Forge.R-project.org")`
4. An example dataset is provided in USLcalc-data.txt.

Download the R software, if you don't already have it.

4 Presentations

Here is a list of my presentations and publications demonstraing how the USL has been applied.

4.1 DSconf 2019

Distributed Systems Conference

Pune, India, February 16, 2019

Applying The Universal Scalability Law to Distributed Systems

When I originally developed the Universal Scalability Law (USL), it was in the context of tightly-coupled Unix multiprocessors [Gunther 1993], which led to an inherent dependency between the serial contention term and the data consistency term in the USL, i.e., no contention, no coherency penalty. A decade later, I realized that the USL would have broader applicability to distributed clusters if this dependency was removed [Gunther 2007]. In this talk I will show examples of how the most recent version of the USL, with three parameters α, β, γ, can be applied as a statistical regression model to a variety of large-scale distributed systems, such as Hadoop, Zookeeper, Sirius, AWS cloud, and Avalanche DLT, in order to quantify their scalability in terms of numerical concurrency, contention, and coherency values.

Video presentation

4.2 ACM 2018

San Francisco Bay ACM Chapter, November 14, 2018

The Data Analytics of Application Scaling and Why There Are No Giants

Like the 30 ft. giant in "Jack and the Beanstalk," myths and fallacies abound regarding application scaling. Many blog posts show some performance data as time-series charts, but otherwise only offer a qualitative analysis. This talk is intended to remedy that by showing you how to QUANTIFY scalability. Time series are not sufficient to assess cost-benefit of cloud services and other scalability trade-offs. After reviewing the nonlinear constraints on the scalability of giants, we apply similar nonlinear data-analytics techniques to determine the universal scalability constraints on such well-known applications as: MySQL, Memcache, NGINX, Zookeeper, and Amazon AWS.

Presentation slides
Video

4.3 ACM 2015

Communications of the ACM, Vol. 58 No. 4, Pages 46-55, 2015

Hadoop Superlinear Scalability [Hadoop 2015]

"We often see more than 100% speedup efficiency!" came the rejoinder to the innocent reminder that you cannot have more than 100% of anything. This was just the first volley from software engineers during a presentation on how to quantify computer-system scalability in terms of the speedup metric. In different venues, on subsequent occasions, that retort seemed to grow into a veritable chorus that not only was superlinear speedup commonly observed, but also the model used to quantify scalability for the past 20 years—Universal Scalability Law (USL)—failed when applied to superlinear speedup data.

Indeed, superlinear speedup is a bona fide phenomenon that can be expected to appear more frequently in practice as new applications are deployed onto distributed architectures. As demonstrated here using Hadoop MapReduce, however, the USL is not only capable of accommodating superlinear speedup in a surprisingly simple way, it also reveals that superlinearity, although alluring, is as illusory as perpetual motion.

ACM Queue
Video at CACM

4.4 Usenix LISA 2014

Seattle, WA: November 9–14, 2014

Super Sizing Your Servers and the Payback Trap

As part of IT management, system administrators and ops managers need to size servers and clusters to meet application performance targets; whether it be for a private infrastructure or a public cloud. In this talk I will first establish an analytic framework that can quantify linear, sublinear and negative scalability. This framework can easily be incorporated into Google Docs or R. Several examples including PostgreSQL, Memcached, Varnish and Amazon EC2

scalability will then be presented in detail. The lesser known phenomenon of superlinearity will be examined using this same framework. Superlinear scaling means achieving more performance than the available capacity would be expected to support.

Slides and video

4.5 Hotsos 2011

Hotsos ORACLE performance symposium
Irving, TX: March 6–10, 2011

Brooks, Cooks and Response Time Scalability

Successful database scaling to meet service level objectives (SLO) requires converting standard Oracle performance data into a form suitable for cost-benefit comparison, otherwise you are likely to be groping in the dark or simply relying on someone else's scalability recipes. Creating convenient cost-benefit comparisons is the purpose of the *Universal Scalability Law* (USL), which I have previously presented using transaction throughput measurements as the appropriate performance metric. However, Oracle AWR and OWI data are largely based on response time metrics rather than throughput metrics.

In this presentation, I will show you how the USL can be applied to response time data. A surprising result is that the USL contains Brooks' law, which is essentially a variant of the old *too-many-cooks* adage: hiring more cooks at the last minute to ensure the meal is prepared on time can cause the preparation to take longer. From the standpoint of the USL, this kind of delay inflation arises from two unique interactions in the kitchen: group conferences and tête-à-têtes between the experienced cooks and the rookie cooks. Replace cooks with Oracle *average active sessions* (AAS) and similar response-time inflation can impact your database SLOs. The USL reveals this effect in a numerical way for easier analysis.

Examples of applying the USL to Oracle performance data will be used to examine Brooks' law and underlying concepts such as delay, wait, latency, averages and response time percentiles.

Presentation slides

4.6 Surge 2010

Surge 2010: The Scalability and Performance Conference
Baltimore, MD, Sep 30–Oct 1, 2010

Quantifying Scalability FTW

You probably already collect performance data, but data ain't information. Successfull scalability requires transforming your data so that you can quantify the cost-benefit of any architectural decisions. In other words:

```
measurement + models == information
```

So, measurement alone is only half the story; you need a method to transform your data. In this presentation I will show you a method that I have developed and applied successfully to large-scale web sites and stack applications to quantify the benefits of proposed scaling strategies. To the degree that you don't quantify your scalability, you run the risk of ending up with WTF rather than FTW.

Speaker list

4.7 Velocity 2010

Accepted title and abstract:

Hidden Scalability Gotchas in Memcached and Friends*

Neil Gunther (Performance Dynamics), *Shanti Subramanyam* (Oracle USA), *Stefan Parvu* (Oracle Finland)

Most web deployments have standardized on horizontal scaleout in every tier: web, application, caching and database; using cheap, off-the-shelf, white boxes. In this approach, there are no real expectations for vertical scalability of server apps like memcached or the full LAMP stack. But with the potential for highly concurrent scalability offered by newer multicore processors, it is no longer cost-effective to ignore their underutilization due to poor, thread-level, scalability of the web stack. In this session we show you how to quantify scalability with the Universal Scalability Law (USL) by demonstrating its application to actual performance data collected from a memcached benchmark. As a side effect of our technique, you will see how the USL also identifies the most signficant performance tuning opportunities to improve web app scalability.

* This work was performed while two of us (S.S. and S.P.) were employed by Sun Microsystems, prior to its acquisition by Oracle Corporation.

Cite as:

```
N. J. Gunther, S. Subramanyam, and S. Parvu.  ‘‘Hidden Scalability Gotchas
in Memcached and Friends.’’  In VELOCITY Web Performance and Operations Conference,
Santa Clara, California, O’Reilly, June, 22--24 2010.
http://velocityconf.com/velocity2010/public/schedule/detail/13046
```

Presentation slides.

Blog post about being accepted for the Velocity 2010 conference Web Ops track.

My review of the conference.

4.8 Ignite! 2009

Ignite! Velocity (San Jose).

Scalability for QuantHeads: How to Do It Between Spreadsheets

Why are there no 30 ft giants like the one in Jack and the Beanstalk"? Why are there no 2,000 ft(?) beanstalks, for that matter?

Engineering weight-strength models tell us their scalability reaches a critical point and they would BOTH collapse. How big can your web site scale before a performance collapse? How do you know? Can you prove it?

Measurement alone is NOT sufficient. How do you know those data are correct? You need both: **Data + Models == Insight**. I'll show you how to prove it using simple spreadsheets. This technique leads to the concept of scalability zones.

5 Bibliography

[Gunther 1993] N. J. Gunther, "A Simple Capacity Model of Massively Parallel Transaction Systems," CMG National Conference, 1993 (PDF)

[Gunther 1998] N. J. Gunther, *The Practical Performance Analyst*, McGraw-Hill, 1998 (Second printing, iUniverse 2000)

[Gunther 2002] N. J. Gunther, "A New Interpretation of Amdahl's Law and Geometric Scalability," arXiv 2002

[Gunther 2007] N. J. Gunther, *Guerrilla Capacity Planning*, Springer 2007

[Gunther 2008] N. J. Gunther, "A General Theory of Computational Scalability Based on Rational Functions," arXiv 2008

[Gunther 2018] N. J. Gunther, USL Scalability Modeling with Three Parameters, The Pith of Performance blog, May 20, 2018

[HoltGun 2008] J. Holtman and N. J. Gunther, "Getting in the Zone for Successful Scalability," arXiv

[Hadoop 2015] N. J. Gunther, P. Puglia and K. Tomasette, "Hadoop Superlinear Scalability: The perpetual motion of parallel performance," Communications of the ACM, Vol. 58 No. 4, Pages 46-55, 2015

[SF ACM 2018] N. J. Gunther, The Data Analytics of Application Scaling and Why There Are No Giants SF Bay ACM Meetup, Walmart Labs, Sunnyvale, California, November 14, 2018

[Wiley 2017] N. J. Gunther and S. Subramanyam and S. Parvu, "A Methodology for Optimizing Multithreaded System Scalability on Multicores," in *Programming Multi-core and Many-core Computing Systems*, eds. S. Pllana and F. Xhafa, Wiley Series on Parallel and Distributed Computing